AI FAQ ASSISTANT API

Naan Mudhalvan Project

AI-Powered FAQ and Customer Support Automation

Team ID : SWTID-2026-6341

Team Size : 5

Team Leader : SAPARNA S

Team Member : SOBITHA K

Team Member : SUNSHRI S R

Team Member : VIDYA K

Team Member : YOGAARASI S P

Project Description

A robust, secure, and intelligent REST API platform designed to streamline FAQ creation, user authentication, and AI-powered content management.

Built with Node.js and Express.js using MongoDB with Mongoose ODM.

JWT authentication and bcrypt password hashing provide secure access control.

Google Gemini AI supports automated FAQ generation, semantic search and dynamic context parsing.

Centralized error handling improves reliability and prevents database leakage.

Scenario-Based Case Study

Background

Priya is a customer support operations manager handling a large backlog of repetitive user inquiries.

Problem

- Difficult to maintain up-to-date FAQ documentation
- Limited visibility into semantic search and query resolution
- Time-consuming manual drafting of complex support answers
- Lack of unified permission layers for private documentation

Solution

- Centralized secure API wrapper
- Verification matrices for content and database operations
- Automated text inference and content creation services

Outcome

- Reduced operational delays
- Schema validation maintains data integrity
- Secure author-based document modification
- Fast FAQ discoverability through indexing

Software & Hardware Requirements

Software

- Windows 10/11, macOS, or Linux
- Node.js v18 or above
- npm v9 or above
- Express.js v4/v5
- MongoDB / Mongoose ODM
- Postman / ThunderClient
- Visual Studio Code

Hardware

- Intel Core i5 8th Gen+ / AMD Ryzen 5 or equivalent
- RAM: 8 GB minimum, 16 GB recommended
- Storage: 1 GB available

Epic-1: Project Architecture

- API Interface – Postman / API tools
- Express Server Gateway – GET, POST, PUT, DELETE, JSON and CORS
- Authentication Middleware – token validation and authorization
- Granular Services – isolated domain/business rules
- Database Interface – Mongoose with MongoDB
- AI Flow – Google Gemini SDK integration
- Layered backend architecture separates request handling, services and data access

Entity-Relationship Design

User Entity

- _id, name, email, hashed password, role, createdAt, updatedAt

FAQ Entity

- _id, question, answer, category, createdBy, createdAt, updatedAt

AI Generation Entity

- topic, generatedQuestion, generatedAnswer, generatedCategory, generatedAt

Category Entity

- categoryName, description, totalFAQs

Key Features

- JWT Authentication & Authorization
- AI FAQ Generation using Google Gemini
- Semantic FAQ Search
- Complete CRUD Operations
- Category Management
- Password Encryption using bcrypt
- Centralized Error Handling
- Mongoose Schema Validation
- RESTful API Architecture
- MongoDB Integration

Roles and Responsibilities

Admin

- Manage users, FAQs, categories and AI-generated content

Content Creator

- Create, edit and delete own FAQs
- Generate FAQs using AI
- Organize FAQs by category

Authenticated User

- View, search and generate AI-assisted answers

Public User

- View and search published FAQs
- Cannot modify FAQ records

User Flow & MVC Architecture

User Flow

1. Register / Login → 2. JWT Token → 3. Token Verification → 4. Authorized API Access → 5. FAQ / AI Operations

Model Layer

Defines Mongoose schemas and data structure.

Controller Layer

Processes requests, validates inputs and delegates business logic.

View / Routing Layer

API routing layer connects requests to controller handlers in this headless backend.

Epic 2: Project Setup and Configuration

1. Create the project folder and src directory.
2. Open the project in VS Code.
3. Initialize the project using npm init -y.
4. Install dependencies:

npm install express mongoose dotenv cors jsonwebtoken bcrypt @google/genai

The project follows a modular backend folder structure for routes, controllers, middleware, models and services.

Epic 3: Backend Development

Core system components:

- Gateway Server (src/server.js)
- Express Setup (src/app.js)
- Security Access Middleware (authMiddleware.js)
- Centralized Error Handler (errorMiddleware.js)

The server loads environment variables, connects to MongoDB, validates authentication tokens and returns standardized API responses.

Epic 4: Database Configuration

Mongoose Schema Definitions

- User schema enforces profile field validation and password hashing.
- FAQ schema stores question, answer, category and creator reference.
- MongoDB provides NoSQL document storage.
- Mongoose ODM maps application models to MongoDB collections and validates data before persistence.

Epic 5: API Testing & Execution

- Install Postman or ThunderClient.
- Start the backend using npm run dev.

API Endpoints

POST /api/auth/register – Public registration
POST /api/auth/login – Public login
POST /api/faqs – Private FAQ creation
GET /api/faqs/search?q=configure – Public search
POST /api/ai/generate-faq – Private AI FAQ generation

API testing verifies authentication, FAQ CRUD/search operations and AI content generation.

AI FAQ Generation

Endpoint: POST /api/ai/generate-faq

Access: Private (Bearer JWT Token)

Input: Topic for FAQ generation

Google Gemini AI generates an FAQ pair containing a question, answer and category-related content.

Generated content can later be saved into the FAQ collection.

Conclusion

AI FAQ Assistant API is a secure, layered backend for FAQ and customer-support automation.

JWT authentication and role-based authorization protect private resources.

Google Gemini AI supports automated FAQ generation and intelligent content workflows.

MongoDB and Mongoose provide structured, validated NoSQL data storage.

The modular RESTful architecture is designed for maintainability and scalability.

Thank You